Can I use machine learning to produce a trading algorithm that can be a suitable replacement to storing cash in a bank

Abstract
This project explores whether it is feasible for the average person in the uk to grow their savings more effectively through algorithmic stock trading compared to traditional bank savings accounts due to the decreasing value of money when stored in bank accounts due to inflation (Bank of England, 2024). Using a Hidden Markov Model (HMM)-based strategy, the algorithm predicts market regimes (bullish, bearish, or sideways) based on technical indicators such as RSI, volatility, volatility of volatility, and momentum. A total of 500 randomly generated parameter combinations were backtested on Apple (AAPL) stock data from 1995 to 2014, with the best strategies evaluated using metrics like Sharpe ratio, cumulative equity return, average return, and maximum drawdown.

Introduction

In recent years, people who have been saving their money in saving accounts have been losing significant value due to low interest rates. According to the Bank of England's website, the median interest rate since 1975 is approximately 9.1%, yet high street savings accounts typically offer much less—often up to 4.75% AER (@moneycouk, 2025). While banks are often seen as "100% secure" (NS&I, 2020) and with "£85,000 per person or £170,000 for joint accounts." Covered for by the FSCS. Though, the highest average savings amount in the uk (in London) is £28,978.40 (Raisin.co.uk, 2024). It is clear that the average person's savings don't even touch the limit covered for banks, which can explain the common notion that banks are one of the safest places to store money, "most common places people choose to store their money"

where your money does not depreciate and is secure, however this can be far from the truth. This raises the question: is there a more effective way for the average person to grow their money? The stock market has historically delivered better long-term returns, but most individuals lack the expertise or time to trade effectively. Algorithmic trading offers a promising alternative, through it being executed in an automated, hands-off fashion. Due to this automated nature, people can benefit from the market without needing to become financial experts as they do not need to devote much time to learning when to buy and sell a stock and also researching different companies. I investigate which machine learning techniques can be used to develop a strategy that reduces volatility and increases returns, making it suitable as a long-term savings solution.

2. Research & Planning

Paul Willmott's (2007) book introducing quantitative finance acts as a self-teaching resource for quantitative finance, building from the ground up by introducing European and American options and then going deeper by covering

topics like stochastic calculus and then the Black-Scholes equation, which is used for pricing derivatives, these are contracts based on an underlying asset. This book was intended more for practical use rather than for academic use. This has meant that there have been relatively few citations but there have been roughly 47 5-star reviews. Its content is more mathematical compared to other practical books like Ernest Chan's book on algorithmic trading. This has meant both books have complimented each other well, with Willmott's book teaching the prerequisites to properly utilise Chan's book.

The original plan was to explore quantitative finance, starting from Paul Wilmott's book on quantitative finance (see appoint 11 for evidence) and Ernest Chan's book on algorithmic trading (see appendix 12 for evidence), followed by niche algorithmic strategies and eventually machine learning applications. However, the complexity and depth of Wilmott's book consumed much more time than expected, due to the complex mathematical ideas introduced. In hindsight, I would have found it better to first delve into the mathematical concepts themselves before seeing their applications in intricate texts like Paul Wilmott's. This learning curve has led me to consider less conceptually challenging and practically complex methods due to time constraints of this project.

The complex mathematical nature of the financial textbooks lead me to look at more machine learning driven sources such as Michael Nielsen's book on neural networks. This was the best resources for helping me understand what is really going on. It was not as heavy on the math and relied more on building the intuition which helped me understand the math when it came to understanding it. I managed to program a simple neural network by just using the 4 equations given in chapter 2(see appendix 2 for the code). I originally found this source from watching 3B1B's series on neural networks, which also helped massively in giving a good intuition behind how neural networks learn and predict. The 2 sources worked well with each other, but I think that seeing 3B1B's series before Michael Nielsen's book was much more beneficial in my understanding. This is due to the easy to digest and slow-paced nature of 3B1B. This confidence led me to look deeper into the more mathematically rigorous resources such as Stanford's CS229 lecture on machine learning, specifically the 2020 edition taught by Andrew Ng. However, upon watching the first lecture, I quickly realised that the pace of the class was much faster and assumed much more prerequisite knowledge than I was comfortable with (see appendix 12 for my notes on this first lecture).

I had also looked at some practical sources, to elevate my programming and machine learning knowledge, to be able to implement the strategy I had envisioned. This included taking a course on python for trading (see appendix 1 for evidence) which helped me understand how to make sense of financial data by utilising libraries like NumPy, Pandas and Matplotlib to plot graphs of distinct factors and see how they relate.

3. Methodology

Initially, I envisioned a sentiment analysis approach which would look at the correlation between the sentiment of 1 company and its stock price. This came about due to my exploration of news articles which put the cause of many price swings to the sentiment of a company, such as if a certain company was hit with a major lawsuit or if their CEO said

something controversial. However, it was proving to be quite difficult to implement. After plotting 1 graph of sentiment over time, I noticed It only showed 1 months' worth of sentiment (see appendix 3 for graph of sentiment against time). I knew that this would not be enough data, considering it is often not possible to gather sentiment of a company for a day due to no news articles being published. Upon researching the API I was using (AlphaVantage) it turned out I could only get 1 request a day which included 1 months' worth of articles on a company. This led me to look at other alternatives such as NewsAPI and Gnews, however, they shared the same downside. The time taken to come to this conclusion, which included scouring the internet for free APIs that gave more than 1 months' worth of articles, that this was not a workable path without spending money, which went against my core aim of not spending money.

Through a video on Markov models on my YouTube feed by "normalised nerd" I discovered the concept of Markov models and how they can predict what might happen next based only on what is happening right now. This concept sounded like it had huge potential due its similarity with how the stock market works. Upon further research on the math behind Markov models on GeeksforGeeks, which looked into algorithms used like Virterbi and Wield-Bauch, there was a library called Hmmlearn which has some of the capabilities of Markov models. How Markov models in my algorithm would work is that they would look at the observed states (the prices, and parameters like RSI and momentum) and look at the probability of one of the 3 states occurring next (bull, bear or sideways market) based on the probability of being in the state it is in.

The final system was implemented in Python using Google Colab. This was because it had built in libraries like NumPy and pandas, which helped significantly when setting up. Also, as it was online, I was able to program anywhere. However, as it was online it made use of sessions whereby if you were inactive for a certain length of time it would close your session. This meant that dependencies that had been preinstalled like Hmmlearn had to be reinstalled each time. It also meant I had to redownload the stock price data of apple each time as often Yahoo finance API was throttled.

There were only 2 languages I was considering for this project, C++ and python. This is because they have the biggest support for programming trading algorithms and have a lot of resources to learn how to use them. However, I picked python in the end over C++, this is because of the following reasons:

- It had better support for machine learning
- I was programming an inter-day trading algorithm so execution speed was not of the essence (if it was then I would have chosen C++ as it is much faster than python)
- I have prior experience with it. This meant I could learn more advanced features quickly (such as lambda functions which is a way of declaring functions on 1 line)

One key aspect of this algorithm was how often it would trade. If this were to be trading on real brokerage accounts, there would be commissions that would have to be made for each trade. For this case being that the average person may not have enough money to pay frequent transaction (as in multiple times in an hour), this is why I think trading inter-day and perhaps inter-week would be most beneficial depending on the risk tolerance.

Another key aspect is the asset class. Larger companies tend to be less volatile due to their less volatile revenue and constant ownership. One of the main goals for the average person is to reduce volatility as much as possible and preferably similar to bank levels. This is why I am choosing big companies like apple.

4. Development

Initial development began by comparing programming environments. Local IDEs like VSCode offered control, but Google Colab was ultimately chosen for its ease of use, pre-installed libraries, and remote accessibility.
APIs tested for data and news sentiment included AlphaVantage, Yahoo Finance, NewsAPI, and GNews. Issues like IP-based request throttling on Yahoo Finance were circumvented by downloading data locally.
Sentiment analysis was performed using FinBERT, but results lacked statistical significance. This led to the transition to Hidden Markov Models. HMMs provided a clearer framework by using historical indicators to estimate the probability of being in a particular market regime, then using these probabilities to drive trading decisions.

Iteration 1- making the sentiment analysis model:

I first decided to plot a graph of how changes in sentiment changed with prices (appendix 4) (code in appendix 5).
The following 2 conclusions can be made from this graph, either there is something wrong with the data, which is affecting the calculations, or that there is no correlation between sentiment and stock price in apple. The former is more likely but given that we are using Finbert, a sentiment analysis machine learning model, it is not feasible to look into how Finbert is exactly calculating the scores. I decided to investigate this further though by looking at a 3d graph of the change in price, sentiment and time (appendix 6). From this it can be clearly seen that there is no correlation, so the data must be wrong.

Iteration 2- making the Hidden Markov model.

This time I decided to use the Hmmlearn library to predict what market regime each day is in. This is done by inputting the adjusted closing prices, RSI window, volatility window, the volatility of the volatility window and momentum window. Here are the reasons I chose these parameters:

Why Hidden Markov model?

One of the key aspects of Hidden Markov models is that they only require limited data to produce accurate predictions, through using transition matrices. This is clearly one of the best solutions then to our problem of data quantity and quality. Competitors to Hidden Markov models include Neural networks, which require a tremendous amount of accurate data to give similar predictions. It is therefore a no-brainer that Hidden Markov models will best work with our current situation.

Adjusted closing prices:

Often when companies issue dividends or announce stock splits the price of the share will go down accordingly. This is not because of the company underperforming or anything

negative, it just represents that the company has a lower market value due to giving out cash in dividend scenario, or that each stock is now worth n times less as there are n times more share.

Why did I choose this indicator?

Often false the Hidden Markov Model can produce false signals if prices drop drastically. Using this indicator ensures that any rapid changes in price are not due to dividends or dilution of shares

Volatility window:

Volatility measures how quickly a stock's price changes over time. The window for which you measure volatility therefore matters a lot, as over a long period you may see much less volatility than over a shorter period, so it is important to get the right window to be able to calculate a volatility dependant on the stock, i.e. for smaller stocks you may want a smaller volatility window due to large fluctuations in price daily whereas for larger stocks you may want a larger volatility window as prices tend to not fluctuate as much daily for larger companies

Volatility of volatility:

This measures how quickly the volatility changes, this can tell you if the volatility is increasing or decreasing which can tell you how unpredictable a stock may be becoming.

Momentum:

This is how quickly a stock's price changes in a certain direction. Unlike volatility, which accounts for both directions in price changes, momentum calculates the change in price in a certain direction.

Why did I choose these indicators:

Research has shown that volatility can be helpful in determining the state of a market Prakash et al. (2022) and show that volatility clustering can be algorithmically partitioned into regime states (Bucci & Ciciretti ,2021). Momentum and RSI have demonstrated regime sensitivity with the momentum strength varying significantly in calm versus turbulent markets (Cao and Copeland, 2022), and RSI serves as a trend-confirming indicator in markets (Segal, 2021) Additionally, it can be seen that combining volatility and RSI significantly improves market regime identification (Business101pub.com, 2025).

How the algorithm works

Without in-depth financial and trading knowledge, it is hard to tell what the optimal windows should be for calculating each parameter. This is why I am using a trial-and-error method by generating random combinations of the parameters and then backtesting them with the hidden Markov model.

The hidden Markov model works by calculating the transition probabilities of observed states to try and calculate the next state. In this scenario we are giving the hidden Markov model, RSI, Momentum, Volatility and the volatility of volatility and the stock price. It then calculates the probability of the market regime being a "bull", "bear" or sideways depending on what the observed state is. One of the key things of Markov models is that it only uses the observed state (i.e. today's data) to predict tomorrow's regime. Also, given that we are using the adjusted closing price, the Markov model will predict tomorrow's closing price. Therefore, as we cannot buy the closing price, we are forecasting 2 days into the future, if the Markov model predicts a bull market regime we will buy the stock at that price, if it predicts a bear market regime we sell all stock. If it is sideways then we do not do anything. One of the key aspects which my model does not currently account for is trading commissions which may be taken by brokerage accounts. (See appendix 7 for code)

I then backtest these different combinations of parameters on Apple's stock data from 1996 to 2014. This helps the hidden Markov model learn the probabilities given the different parameters. It will then plot a graph of equity return as a cumulative percentage from the starting stock price at which the model first bought the stock (see appendix 8 to see a screenshot of the different graphs produced)

After plotting each of the 500 parameter combinations on an equity against time curve I will look at the best preforming strategies by calculating the Sharpe ratio, maximum drawdown and average return. This will help me identify the best 20 strategies upon which I will then backtest these strategies on unseen data in which the Markov model cannot change the values of its transition probabilities depending on the new values given (see appendix 9 for an example of one of the graphs). The reason for this test on unseen data is to see if the Markov model has found a generalised set of transition probabilities which would make its predictions more reliable. It also checked whether the windows set for the different

parameters perform as well as they did on the previous batch on data. If they do then that would be a good indicator of those windows being fit for a certain company, in this case apple.

5. Evaluation

Each of the 500 parameter combinations was evaluated by plotting the equity curve. The best-performing strategies will be assessed on:

- **Sharpe Ratio**: Measures risk-adjusted return
- **Cumulative Equity Return**: Overall profitability of the strategy.
- **Maximum Drawdown**: Largest historical loss from a peak.

(see appendix 8 for picture of the multiple back tests performed)

The initial results show strong potential as most back tests produced positive cumulative equity returns of 100 times its starting value. I will look to add the aforementioned metrics in the future and make a decision of which parameters to backtest based on these metrics. In order to reimplement the Markov model I have encapsulated the algorithm inside of a function so that it can be used when backtesting on seen data.

In order to rank each parameter combination, I calculated each metric (Sharpe ratio, final equity and drawdown) as it was backtested in the for loop. This made the code take a considerable amount of time to execute (4 minutes according to google Colab), though this is expected given the sheer volume of data we are calculating with.

(See appendix 16 and 17 for the updated code and the output)

From appendix 17 we can see the 10 best performing parameter combinations in terms of Sharpe ratio (higher is better), maximum drawdown (lower is better) and cumulative equity return (higher is better). Though, I have not been able to extract the exact values of these metrics, they are not the most important thing to worry about right now as this is seen data so it may be that these parameters have overfitted to give the best metrics. This will be exposed and the parameters with real potential will become apparent once we test on unseen data.

6. Limitations

- I have not included portfolio-level diversification — tested on AAPL only.
- No live deployment or real-time data integration. I have not back tested on Realtime data on a paper trading account so this may not be an accurate reflection yet
- HMM assumes stationary regimes by just giving the probability that the market will be of a certain regime, which may not reflect reality.
- I had to discard the Sentiment model due to poor signal-to-noise ratio and lack of quality data.

7. Conclusion

This artefact does demonstrate that there is potential for algorithmic trading to be a suitable alternative for people to invest in, in order to not have their cash depreciate. While the

artefact has not been tested on real-time data just yet, the foundational concepts show room for growth. With more rigours testing, portfolio optimisation and integrating live data, this strategy could become a low-volatility tool for long-term growth of cash above the rate of inflation for the ordinary person. The benefit of this algorithm executing automatically means there is low maintenance for consumers and so this can be mass distributed to people at a low cost. This also improves the returns of the algorithm if many people invest as algorithmic trading typically works best with large volumes of capital.

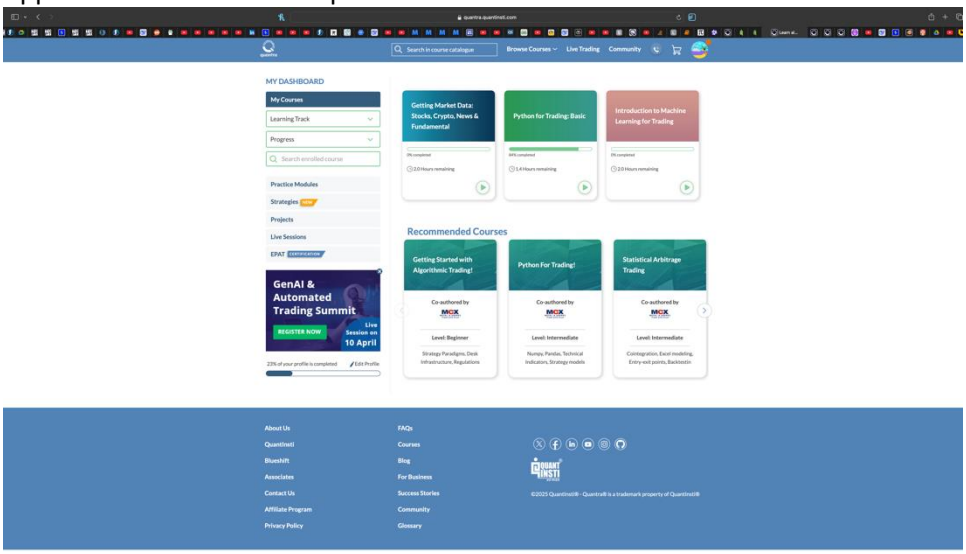
8. Bibliography

- Chan, E. (n.d.). *Algorithmic Trading: Winning Strategies and Their Rationale*. [pdf] Available at: https://www.myquant.cn/uploads/default/original/1X/4c7037365a4bf1623734c1c899baed7855061ace.pdf [Accessed 3 Dec. 2024].
- CEPR. (2025). *Twitter sentiment and stock market movements: The predictive power of social media*. [online] Available at: https://cepr.org/voxeu/columns/twitter-sentiment-and-stock-market-movements-predictive-power-social-media[Accessed 8 May 2025].
- Clenow, A.F. (2015). *Stocks on the Move: Beating the Market with Hedge Fund Momentum Strategies*. [online] Available at: https://www.amazon.co.uk/Stocks-Move-Beating-Momentum-Strategies/dp/1511466146 [Accessed 3 Dec. 2024].
- DATAVERSITY. (2023). *Types of Machine Learning and Their Applications*. [online] Available at: https://www.dataversity.net/types-of-machine-learning-and-their-applications/ [Accessed 19 Aug. 2023].
- Editor. (2019). *The Power of R for Trading (Part 1)*. Macrosynergy. [online] Available at: https://macrosynergy.com/research/the-power-of-r-for-trading-part-1/ [Accessed 8 May 2025].
- Harris, L. (2002). *Trading and Exchanges: Market Microstructure for Practitioners*. [pdf] Available at: https://www.acsu.buffalo.edu/~keechung/MGF743/Readings/Trading-Exchanges-Market-Microstructure-Practitioners%20Draft%20Copy.pdf [Accessed 3 Dec. 2024].
- HuggingFace. (n.d.). *ProsusAI/FinBERT*. [online] Available at: https://huggingface.co/ProsusAI/finbert [Accessed 1 May 2025].
- HMMlearn Documentation. (2025). *API Reference*. [online] Available at: https://hmmlearn.readthedocs.io/en/latest/api.html [Accessed 1 May 2025].
- Investopedia. (n.d.). *Drawdown Definition*. [online] Available at: https://www.investopedia.com/terms/d/drawdown.asp[Accessed 1 May 2025].
- Money.co.uk. (2025). *Instant Access Savings Accounts*. [online] Available at: https://www.money.co.uk/savings-accounts/instant-access-savings [Accessed 8 May 2025].
- Nielsen, M. (2015). *Neural Networks and Deep Learning*. [online] Available at: http://neuralnetworksanddeeplearning.com [Accessed 15 April 2025].
- O'Reilly, D. (2023). *Types of Machine Learning and Their Applications*. DATAVERSITY. [online] Available at: https://www.dataversity.net/types-of-machine-learning-and-their-applications/ [Accessed 19 Aug. 2023].
- Quantocracy. (2015). *Quant Mashup*. [online] Available at: https://quantocracy.com [Accessed 3 Dec. 2024].
- Random House. (2024). Ananthaswamy, A. *Why Machines Learn*.
- Sample Chapter. (n.d.). *Neural Networks*. [pdf] Penguin Books. Available at: https://cdn.penguin.co.uk/dam-assets/books/9780241586488/9780241586488-sample.pdf [Accessed 26 Nov. 2024].

- Wilmott, P. (2007). *Paul Wilmott Introduces Quantitative Finance*. Chichester: Wiley.
- Yahoo Finance. (2025). *Yahoo Finance API Documentation*. [online] Available at: https://yfinance-python.org/reference/api/yfinance.Ticker.html [Accessed 20 April 2025].
- *Prakash, A., James, N., Menzies, M. and Francis, G. (2022). Structural Clustering of Volatility Regimes for Dynamic Trading Strategies. Applied Mathematical Finance, pp.1–39. doi:https://doi.org/10.1080/1350486x.2021.2007146.*
- *Bucci, A. and Ciciretti, V. (2021). Market Regime Detection via Realized Covariances: A Comparison between Unsupervised Learning and Nonlinear Models. [online] arXiv.org. Available at: https://arxiv.org/abs/2104.03667.*
- *Cao, J. and Copeland, L. (2022). Momentum and market volatility: a Bayesian regime-switching model. The European Journal of Finance, pp.1–25. doi:https://doi.org/10.1080/1351847x.2022.2062250.*
- *Segal, T. (2021). Understanding Momentum Indicators and RSI. [online] Investopedia. Available at: https://www.investopedia.com/investing/momentum-and-relative-strength-index/.*
- Raisin.co.uk. (2024). *Average savings by age in the UK (2023)*. [online] Available at: https://www.raisin.co.uk/newsroom/better-saving-money/.
- *NS&I. (2020). How we protect your money. [online] Available at: https://www.nsandi.com/get-to-know-us/security/protect-your-money.*
- *Pettinger, T. (2022). Historical Interest Rates UK. [online] Economics Help. Available at: https://www.economicshelp.org/blog/1485/interest-rates/historical-real-interest-rate/.*

9. Appendices(NB lines which start with a hashtag in code are comments of the code, explaining it line by line)

Appendix 1- screenshot of quanta course



Appendix 2- screenshot of neural network code

```
import numpy as np
import random

class Network:
    def __init__(self, layers):
        self.N_layers = len(layers)
        self.layers = layers
        self.biases = [np.random.randn(y, 1) for y in layers[1:]]
        self.weights = [np.random.randn(y, x) for x,y in zip(layers[1:], layers[:-1])]

    def feedforward(self, inp):
        for w,b in zip(self.weights, self.biases):
            z = np.dot(w.transpose(),inp) + b
            inp = sigmoid(z)
        return inp

    def update(self, mini_batch, Lrate):
        grad_b = [np.zeros(bias.shape) for bias in self.biases]
        grad_w = [np.zeros(weight.shape) for weight in self.weights]
        for test in mini_batch:
            print(f"test data is{test}")
            delta_grad_b, delta_grad_w = self.backpropagation(test[0], test[1])
            for w,b in zip(range(len(self.weights)), range(len(self.biases))):
                grad_b[b] = grad_b[b] + delta_grad_b[b]
                grad_w[w] = grad_w[w] + delta_grad_w[w]
        for w,b in zip(range(len(self.weights)), range(len(self.biases))):
            self.weights[w] = self.weights[w] - Lrate/len(mini_batch)* grad_w[w]
            self.biases[b] = self.biases[b] - Lrate/len(mini_batch)* grad_b[b]

    def learning(self, batch, Lrate, Runs):
        for run in range(Runs):
            np.random.shuffle(batch)
            if len(batch) < Runs:
                mini_batches = [batch]
            else:
                mini_batch_size = len(batch) // Runs
                print(f"mini_batch_size:{mini_batch_size}")
                mini_batches = [batch[x:x + mini_batch_size] for x in range(0, Runs * mini_batch_size,
mini_batch_size)]
            for mini_batch in mini_batches:
                print(f"this is mini batch{mini_batch}")
                self.update(mini_batch, Lrate)

    def backpropagation(self, inp, actualValue):
        mini_grad_b = [np.zeros(bias.shape) for bias in self.biases]
        mini_grad_w = [np.zeros(weight.shape) for weight in self.weights]
        activation = inp
        activations = [inp]
        WeightedInputs = []
        for b, w in zip(self.biases, self.weights):
            print(f"this is the weight matrix{w}")
            print(f"this is the activation value{activation}")
            print(f"this is the bias{b}")
            WI = np.dot(w.transpose(), activation) + b
            WeightedInputs.append(WI)
            activation = sigmoid(WI)
            activations.append(activation)
```

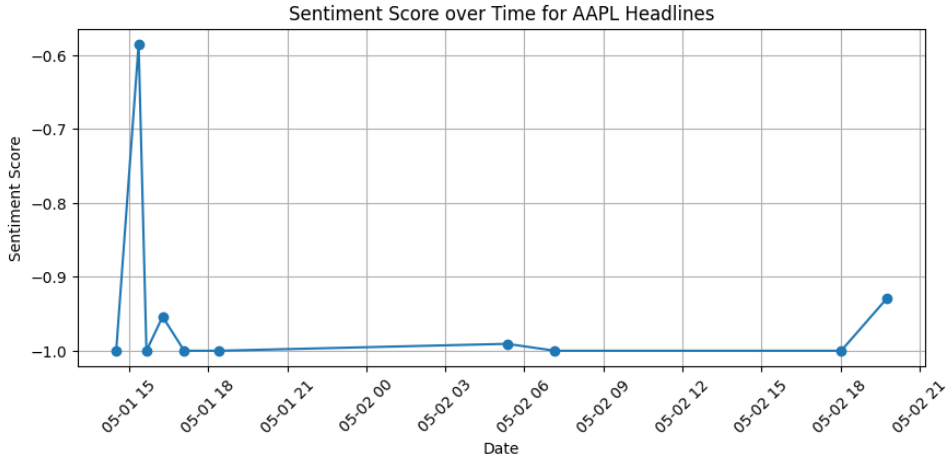
```
        OutputDelta = (activations[-1] - actualValue) * sigmoid_derivative(WeightedInputs[-1])
        for w, b in zip(range(2,len(self.weights), 1), range(2 , len(self.biases), 1)):
            delta = (np.dot((self.weights[-w + 1].transpose()), OutputDelta)) *
sigmoid_derivative(WeightedInputs[-w])
            OutputDelta = delta

            mini_grad_b[-b] =  delta
            mini_grad_w[-w] =  np.dot(delta, activations[-w-1].transpose())
        return mini_grad_b, mini_grad_w

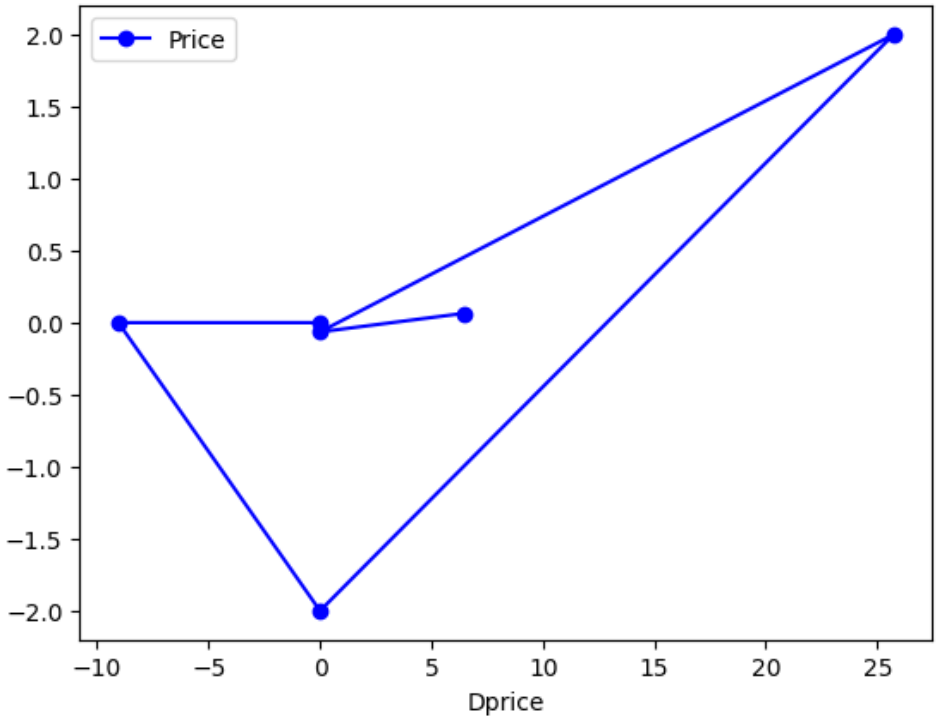
def sigmoid( z):
    return 1.0/(1.0+np.exp(-z))
def sigmoid_derivative(z):
    return sigmoid(z)*(1-sigmoid(z))

# 2-input, 2-output identity mapping
def generate_identity_data(n_samples):
    data = []
    for i in range(n_samples):
        x = random.random()
        y = 2*x
        # FIX: Use 2D arrays (1,1) consistently
        input_vector = np.array([[x]])      # shape (1,1)
        output_vector = np.array([[y]])     # shape (1,1)
        data.append((input_vector, output_vector))
    return data
data = generate_identity_data(100)
net = Network([1, 20, 1])
net.learning(data, Lrate=0.1, Runs=10)
# Test on a sample input
test_input = np.array([[0.5]])
print("Prediction:", net.feedforward(test_input))
```

Appendix 3- of graph of sentiment against time



Appendix 4- change in price against sentiment



Appendix 5- code for sentiment analysis

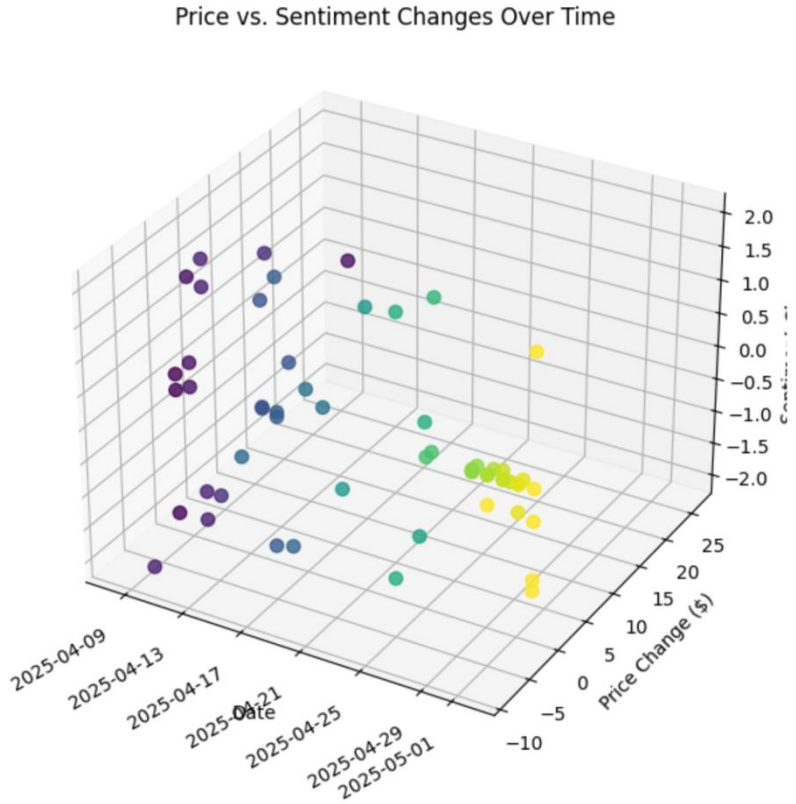
```
import requests
from transformers import pipeline
from transformers import BertTokenizer, BertForSequenceClassification
from transformers import pipeline
import matplotlib.pyplot as plt
import yfinance as yf
from datetime import datetime
import numpy as np
import pandas as pd
import time
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.dates as mdates
from datetime import datetime, timedelta
from google.colab import userdata
#Collecting news articles for corresponding stocks
TickerA = 'TSLA' #abbreviation of company used in stock market
API = userdata.get('NewsAPI')
end_date = datetime.now()
start_date = end_date - timedelta(days=30)
#Making API request for news articles in english for the specific companies sorted by
date published
newsA =
requests.get(f'https://newsapi.org/v2/everything?q={TickerA}&apiKey={API}&from={start_
date}&to={end_date}&pageSize=200"')
#Structuring the data
newsA = newsA.json()
#Formatting price history for respective companies
AAPL = yf.download("AAPL", start=from_date, end=to_date, auto_adjust=True)
AAPL = AAPL.reset_index()
AAPL = AAPL.iloc[1:].reset_index(drop=True)
# Flatten MultiIndex columns
AAPL.columns = ['_'.join(col).strip() for col in AAPL.columns.values]
# Optional: Rename 'Close_AAPL' to just 'Close' for easier merge
AAPL.rename(columns={'Close_AAPL': 'Close', 'Date_': 'Date'}, inplace=True)
print("AAPL data columns:", AAPL.columns)
print(AAPL.head())
AAPL = AAPL.drop(['Open_AAPL', 'High_AAPL', 'Low_AAPL', 'Volume_AAPL'], axis=1)
MSFT = MSFT.drop(['Open', 'High', 'Low', 'Volume', 'Close'], axis=1)
AAPL['Date'] = pd.to_datetime(AAPL['Date'])
#Using prebuilt financial sentiment analysis model to give a sentiment score on each
article in the range between -1(negative) to 1(positive)
model = BertForSequenceClassification.from_pretrained('yiyanghkust/finbert-tone',
num_labels=3)
```

```
tokenizer = BertTokenizer.from_pretrained('yiyanghkust/finbert-tone')
finbert = pipeline("sentiment-analysis", model=model, tokenizer=tokenizer)
#Using prebuilt model to give sentiment score on each article and appending to a results list to plot
def sentiment_score(news):
  result = []
  for article in news:
        try:
            # Get sentiment score
            score = finbert(article['description'])
            # Handle score calculation
            if score[0]['label'] == 'Positive' or score[0]['label'] == 'Neutral':
                score_val = score[0]['score']
            else:
                score_val = -1 * score[0]['score']
            # Append result
            result.append([
                score_val,
                article["publishedAt"][:10]  # YYYY-MM-DD format
            ])
        except Exception as e:
            print(f"Skipping article due to error: {e}")
            continue
  return result
print(f'this is newsA:{newsA}')
articlesA = newsA['articles']
resultA = sentiment_score(articlesA)
print(f'This is Result A:{resultA}.  end of message.')
#merging price and sentiment for stock A where both have the same date so a graph can be plotted of sentiment against price to see if there is a correlation and perhaps causation
Sentiment_A = pd.DataFrame(sentiment_score(articlesA), columns=['Sentiment', 'Date'])
print(f'This is Sentiment A: {Sentiment_A}.  end of message.')
Sentiment_A['Date'] = pd.to_datetime(Sentiment_A['Date'])
AAPL['Date'] = pd.to_datetime(AAPL['Date'])
print(Sentiment_A['Date'].dtype)
print(AAPL['Date'].dtype)
Sentiment_A['Date'] = pd.to_datetime(Sentiment_A['Date']).dt.floor('D')
AAPL['Date'] = pd.to_datetime(AAPL['Date']).dt.floor('D')
print(set(Sentiment_A['Date']) & set(AAPL['Date']))
print(AAPL[['Date', 'Close']].head())
print(Sentiment_A.head())
print(set(Sentiment_A['Date']) & set(AAPL['Date']))
PSA = pd.merge(AAPL[['Date','Close']], Sentiment_A, on='Date', how='inner')
print(f'PSA: {PSA} end of message')
#Finding change in price for corresponding price and sentiment scores to see how much a change in sentiment affects a change in price
delta = []
for i in range(1,len(PSA)):
```

```
  deltaP = float(PSA['Close'][i]) - float(PSA['Close'][i - 1])#change in price
  deltaS = float(PSA['Sentiment'][i]) - float(PSA['Sentiment'][i - 1])#change in Sentiment
score
  Date = PSA['Date'][i]#date when both occured
  delta.append([deltaP, deltaS, Date])
  print(f'this is {i} of delta: {deltaP, deltaS, Date}')
print(f'this is delta :{delta} end of message')
# Convert the delta list to a DataFrame for easier handling
delta_df = pd.DataFrame(delta, columns=['Delta_Price', 'Delta_Sentiment', 'Date'])
print(f' this is delta_df: {delta_df}. end of message')
# Convert dates to numerical values (days since first date) for 3D plotting
# Ensure 'Date' is datetime
delta_df['Date'] = pd.to_datetime(delta_df['Date'])
# First ensure your Sentiment_A and price DataFrames are properly aligned
# Assuming you have:
# Sentiment_A with columns ['Sentiment', 'Date']
# AAPL (price data) with columns ['Date', 'Close']
# Sort by date to ensure proper delta calculation
delta_df = delta_df.sort_values('Date')
print(f'this is delta df:{delta_df}.  end of message')
plt.figure(figsize=(10, 6))
plt.scatter(delta_df['Delta_Sentiment'], delta_df['Delta_Price'], c=delta_df['Date'],
cmap='viridis', alpha=0.7)
plt.colorbar(label='Date')
plt.xlabel('Delta Sentiment')
plt.ylabel('Delta Price')
plt.title('Delta Sentiment vs Delta Price')
plt.show()

from sklearn.linear_model import LinearRegression
# Prepare the data
X = delta_df[['Delta_Sentiment']].dropna()
y = delta_df['Delta_Price'].iloc[X.index]
# Fit the regression model
model = LinearRegression()
model.fit(X, y)
# Predict the values
y_pred = model.predict(X)
# Plot
plt.figure(figsize=(10, 6))
plt.scatter(X, y, label='Data Points', alpha=0.7)
plt.plot(X, y_pred, color='red', label='Regression Line')
plt.xlabel('Delta Sentiment')
plt.ylabel('Delta Price')
plt.title('Delta Sentiment vs Delta Price with Regression Line')
plt.legend()
plt.show()
```

Appendix 6- changes in sentiment against price against time



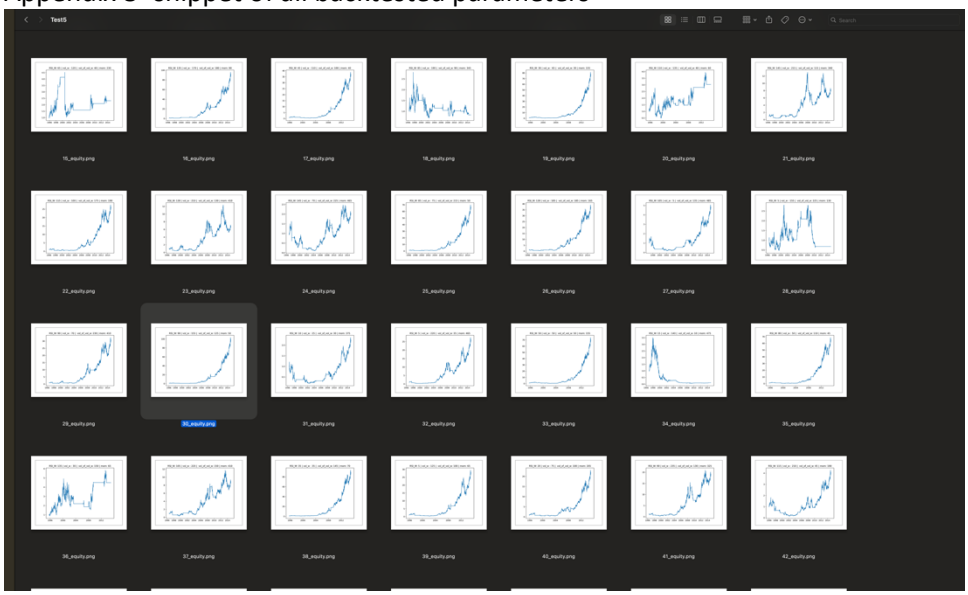
Appendix 7- code for markov model

```
import random
from sklearn.preprocessing import StandardScaler
import pandas as pd
import yfinance as yf
import ta
from ta.momentum import RSIIndicator
import matplotlib.pyplot as plt
from hmmlearn import hmm# Hidden Markov model library
import os
#Collect time series stock data to backtest
#stock_data = yf.download('AAPL', start='1991-01-01', end='2011-01-01',
auto_adjust=False)
stock_data = pd.read_csv('/content/apple.csv')
stock_data.set_index('Date', inplace=True)
stock_data.index = pd.to_datetime(stock_data.index)
# Filter the data to keep only the records between '1995-01-01' and '2015-01-01'
stock_data = stock_data[(stock_data.index >= '1995-01-01') & (stock_data.index <=
'2015-01-01')]
indicators = ['RSI', 'Volatility', 'VolofVol', 'mom']
indicator_data = pd.DataFrame(columns=['Date', 'RSI', 'Volatility', 'VolofVol', 'mom'])
save_dir = '/content/drive/MyDrive/StockreturnAAPl'
RSI_window_range = list(range(5,150,5))
vol_window_range = list(range(5,250,5))
vov_window_range = list(range(5,250,5))
mom_window_range = list(range(5,500,5))
best_runs = []
N = 500
def backtest(N,stock_data, indicator_data, save_dir):
  for i in range(1, N+1):
    if i % 10 == 0:
      print(f"run index is {i}")
    try:
      indicator_data = pd.DataFrame(index=stock_data.index.copy())
      #Calculating technical indicators
      RSI_window = random.choice(RSI_window_range)# 14 day for relative strength index
      vol_window = random.choice(vol_window_range)
      vol_of_vol_window = random.choice(vov_window_range)  # 60 days for volatility of
volatility
      mom_window = random.choice(mom_window_range)
      indicator_data['Date'] = stock_data.index
      indicator_data.set_index('Date', inplace=True)
      indicator_data['RSI'] = RSIIndicator(close=stock_data["Close"],
window=RSI_window).rsi()
```

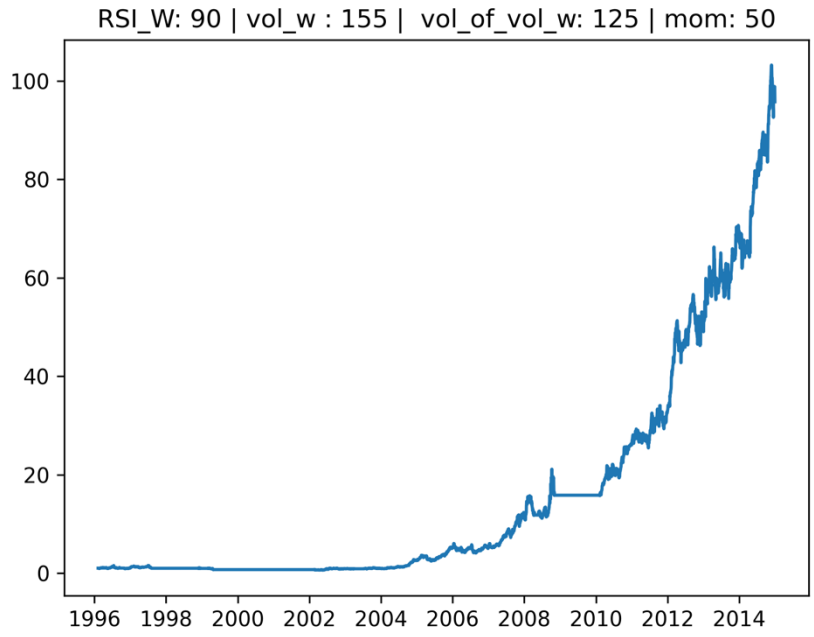
```
        indicator_data['Volatility'] =
stock_data['Close'].pct_change().rolling(window=vol_window).std()
        indicator_data['VolofVol'] =
indicator_data['Volatility'].rolling(window=vol_of_vol_window).std()
        indicator_data['mom'] = stock_data['Close'].pct_change(mom_window)
        indicator_data.dropna(inplace=True) # If there are record which have no value, e.g
momentum for first record then they are dropped
        #3 components,for bullish,bearish and sideways markets respectively, covariance=
full to measure correlation ...
        #between parameters and measure variance of parameters, tol is the percentage
error at which we can stop learning and random state ensures that our results are not
due to pure luck
        scaler = StandardScaler()
        scaled_features = scaler.fit_transform(indicator_data[indicators])
        model = hmm.GaussianHMM(n_components = 3, covariance_type="diag",
n_iter=300, tol=0.001, random_state=50)
        model.fit(scaled_features)
        hidden_states = model.predict(scaled_features)
        #Each regime is essentially a 'state' of a market i.e there is a state for a
bullish,bearish and sideways market
        indicator_data['Regime'] = pd.Series(hidden_states, indicator_data.index).shift(2) #
store predicted regime number for specific day
        indicator_data['Returns'] = stock_data['Close'].pct_change()# show daily returns for
each day
        regime_returns = indicator_data.groupby('Regime')['Returns'].mean()# calculates
average return for each regime, we can use this to attach labels to each regime
        worst_state, best_state = regime_returns.idxmin(), regime_returns.idxmax()
        probs = model.predict_proba(scaled_features)#calculates the probabilties of being in
each regime at each time step(1 day)
        mapping = {regime_returns.iloc[-1]: 'Bull', regime_returns.iloc[0]: 'Bear',
regime_returns.iloc[1]: 'Sideways'}
        indicator_data['Regime_Label'] = indicator_data['Regime'].map(mapping) # adds a
column showing the most likely regime the stock is in at a specific time
        #assigning column label for all the probabilities of each regime at each time step and
appending to the indicator dataframe
        indicator_data['Position'] = 0
        indicator_data['Strat_Return'] = 0
        indicator_data.loc[indicator_data['Regime'] == best_state, 'Position'] = 1
        indicator_data.loc[indicator_data['Regime'] == worst_state, 'Position'] = -1
        indicator_data['Strat_Return'] = indicator_data['Position']*indicator_data['Returns']
        indicator_data['Strat_Equity'] = (1 + indicator_data['Strat_Return']).cumprod()
        plt.plot(indicator_data['Strat_Equity'])
        plt.title(f"RSI_W: {RSI_window} | vol_w : {vol_window} |  vol_of_vol_w:
{vol_of_vol_window} | mom: {mom_window}")
        plt.savefig(os.path.join(save_dir, f'{i}_equity.png'), dpi=300)
        plt.close()
    except:
        print("***ignoring the run as there was an error****")
        continue
```

backtest(N,stock_data, indicator_data, save_dir)

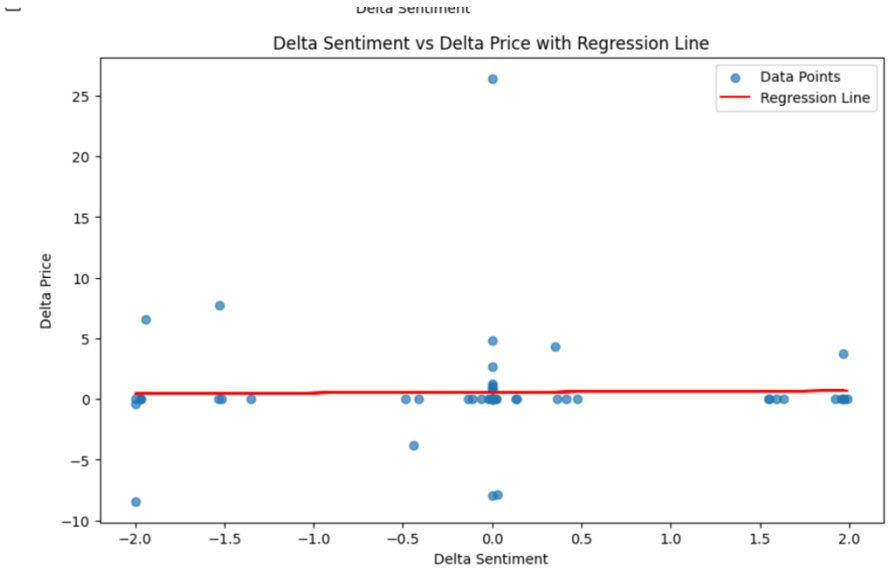
Appendix 8- snippet of all backtested parameters



Appendix 9-snippet of 1 backtested graph



Appendix 10- regression line of sentiment and price



Appendix 11- snippet of notes on PWIQF



Appendix 12- Snippet of notes on algorithmic trading by Ernest chan



Appendix 13- notes in CSS229 by Andrew Ng

Linear regression | 65

Supervised learning
-training set
-feed to learning algorithm
-outputs hypothesis

Data —> learning algo —> a estimated hypothesis

Designing learning algorithm

How do you represent the hypothesis, h?

$h(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots$

Alternative notation

$h(x) = \text{sigma } \theta_i x_i = \sum_{j=0}^{n} \theta_j x_j$
Where $x_0 = 1$

The biases and variables then turn into vectors

θ - parameters

Job of parameter is to choose the parameter

M = number of training samples(#rows in table)
X = inputs/variables
Y = output/target variable

(x,y) training sample

n = # features

How to choose θ?

Choose such that $h(x) \approx y$ for training samples

$h_\theta(x)$
- depends both on θ and x

Want to minimize:
$J(\theta) = \frac{1}{2} \sum_{i=1}^{m} (h(x)-y)^2$

Find the θ such that $J(\theta)$ minimized

Gradient descent (batch gradient descent)

Start with some θ e.g. $\theta = \begin{bmatrix} 0 \\ 0 \\ \vdots \end{bmatrix}$
Keep changing θ to reduce $J(\theta)$
"descend by δ"

$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$ α - learning rate

for each j

$\theta_j := \theta_j - \alpha \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) \cdot x_j^{(i)}$

$\frac{\partial}{\partial \theta_j} J(\theta)$

repeat until convergence
for $j = (0, 1, \dots n)$

$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{\partial}{\partial \theta_j} \frac{1}{2} (h_\theta(x) - y)^2$

$= 2 \times \frac{1}{2} (h_\theta(x) - y) \cdot \frac{\partial}{\partial \theta_j} (h(x) - y)$

$= (h_\theta(x) - y) \frac{\partial}{\partial \theta_j} (\theta_0 x_0 + \theta_1 x_1 \dots \theta_n x_n - y)$

$= (h_\theta(x) - y) \cdot x_j$

Disadvantages of batch gradient descent
- to make 1 update a sum of all training samples must be calculated

Stochastic gradient descent

Repeat {
  For $i = 1$ to m
    $\theta_j := \theta_j - \alpha (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$ } - update summing j

Normal equation

$\theta = (\dots)$

$J(\theta)$: function mapping parameters to real numbers

$\nabla_\theta J(\theta)$ - derivative $\rightarrow \begin{bmatrix} \frac{\partial J}{\partial \theta_0} \\ \frac{\partial J}{\partial \theta_1} \\ \vdots \\ \frac{\partial J}{\partial \theta_n} \end{bmatrix}$

θ - $n+1$ dimensional vector

solve $\nabla_\theta J(\theta) = 0$

you might have a 2×2 matrix $A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$

$f : A \mapsto \mathbb{R}$ (real number)

eg is $f(A) = A_{11} + A_{12}^2$

and $A = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$

$f(A) = 5 + 6^2$

$\nabla_A f(A) = \begin{bmatrix} \frac{\partial f}{\partial A_{11}} & \frac{\partial f}{\partial A_{12}} \\ \frac{\partial f}{\partial A_{21}} & \frac{\partial f}{\partial A_{22}} \end{bmatrix}$ - derivative of matrix has same dimensions

IF $A = \mathbb{R}^{n \times n}$

$\text{tr} A = \sum_i$ diagonal entries $= A_{ii}$

"trace of A"

$\text{tr} A = \text{tr} A^T$

properties of trace function

if $f(A) = \text{tr} AB$ where B is a fixed matrix

$\nabla_A f(A) = B^T$

$\text{tr} BA = \text{tr} AB$

$\nabla_A \text{tr} A A^T C = CA + C^T A$

Remember $J(\theta) = \frac{1}{2} \sum_{i=1}^{m} (h(x^{(i)}) - y^{(i)})^2$

$\nabla_\theta J(\theta) = \nabla_\theta \frac{1}{2} (X\theta - y)^T (X\theta - y)$

$= \frac{1}{2} \nabla_\theta (\theta^T X^T - y^T)(X\theta - y)$

expand then take derivative ...

$= \frac{1}{2} \nabla_\theta [\theta^T X^T X \theta - \theta^T X^T y - y^T X \theta + y^T y]$

$= \frac{1}{2} [X^T X \theta - X^T y - X^T y]$ then set to 0 and solve for θ

$X^T X \theta = X^T y$ "normal equation"

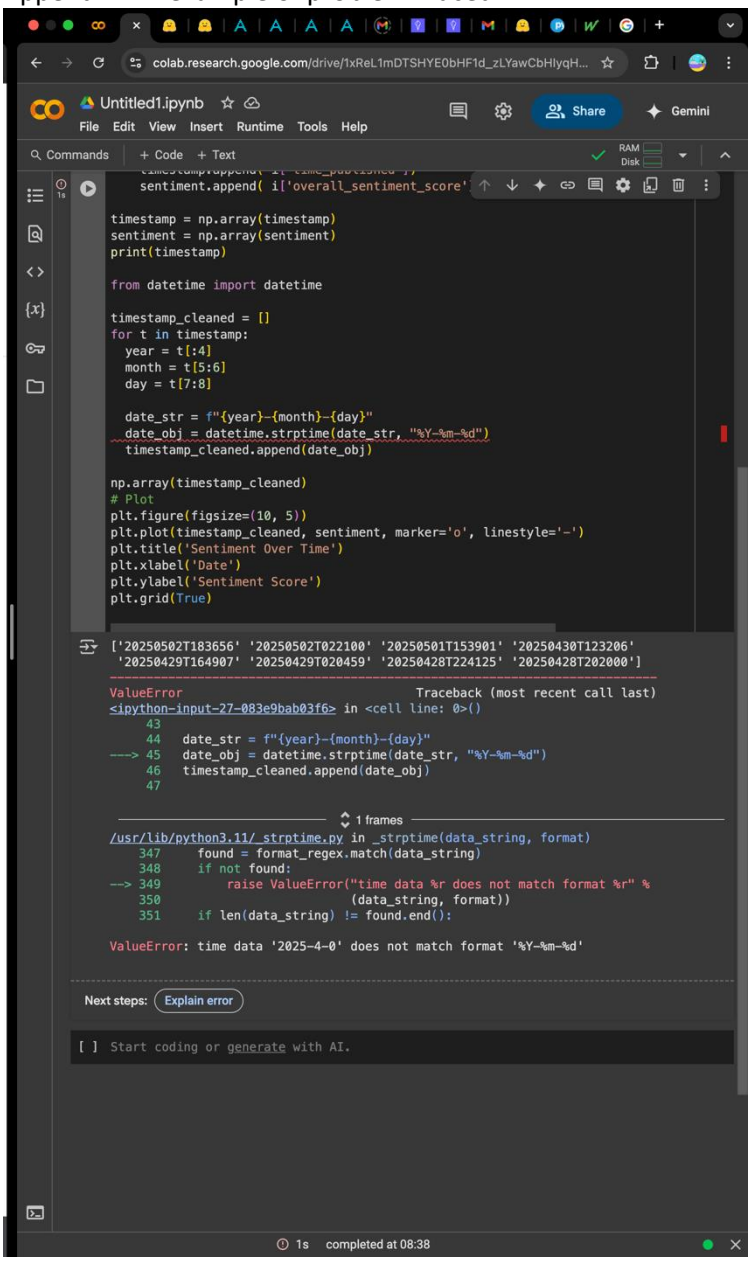
$\theta = (X^T X)^{-1} X^T y$

$X = \begin{bmatrix} \dots & (x^{(1)})^T & \dots \\ \dots & (x^{(2)})^T & \dots \\ & \vdots & \\ \dots & (x^{(m)})^T & \dots \end{bmatrix} \quad \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix} = \begin{bmatrix} x^{(1)} \theta \\ x^{(2)} \theta \\ \vdots \\ x^{(m)} \theta \end{bmatrix}$

$X\theta - y = \begin{bmatrix} h_\theta(x^{(1)}) - y^{(1)} \\ \vdots \\ h_\theta(x^{(m)}) - y^{(m)} \end{bmatrix} \quad y = \begin{bmatrix} h_\theta(x^{(1)}) \\ \vdots \\ h_\theta(x^{(m)}) \end{bmatrix}$

- if X is singular (some have redundant features) use "pseudo-inverse"

Appendix 14- example of problem I faced



Appendix 15- Problems with Yfinance



Appendix 16-Updated code to rank the parameter combinations

```
import random
from sklearn.preprocessing import StandardScaler
import pandas as pd
import yfinance as yf
import ta
from ta.momentum import RSIIndicator
import matplotlib.pyplot as plt
from hmmlearn import hmm# Hidden Markov model library
import os
from collections import defaultdict
#Collect time series stock data to backtest
#stock_data = yf.download('AAPL', start='1991-01-01', end='2011-01-01',
auto_adjust=False)
stock_data = pd.read_csv('/content/apple.csv')
stock_data.set_index('Date', inplace=True)
stock_data.index = pd.to_datetime(stock_data.index)

# Filter the data to keep only the records between '1995-01-01' and
'2015-01-01'
stock_data = stock_data[(stock_data.index >= '1995-01-01') &
(stock_data.index <= '2015-01-01')]
indicators = ['RSI', 'Volatility', 'VolofVol', 'mom']
indicator_data = pd.DataFrame(columns=['Date', 'RSI', 'Volatility',
'VolofVol', 'mom'])
save_dir = '/content/StockreturnAAPL'

RSI_window_range = list(range(5,150,5))
vol_window_range = list(range(5,250,5))
vov_window_range = list(range(5,250,5))
mom_window_range = list(range(5,500,5))
best_runs = []
N = 500

def backtest(N,stock_data, indicator_data, save_dir):
  for i in range(1, N+1):
      if i % 10 == 0:
        print(f"run index is {i}")
      indicator_data = pd.DataFrame(index=stock_data.index.copy())
      #Calculating technical indicators
      RSI_window = random.choice(RSI_window_range)# 14 day for relative
strength index
      vol_window = random.choice(vol_window_range)
      vol_of_vol_window = random.choice(vov_window_range)  # 60 days
for volatility of volatility
      mom_window = random.choice(mom_window_range)
      indicator_data['Date'] = stock_data.index
      indicator_data.set_index('Date', inplace=True)
```

```
      indicator_data['RSI'] = RSIIndicator(close=stock_data["Close"],
window=RSI_window).rsi()
      indicator_data['Volatility'] =
stock_data['Close'].pct_change().rolling(window=vol_window).std()
      indicator_data['VolofVol'] =
indicator_data['Volatility'].rolling(window=vol_of_vol_window).std()
      indicator_data['mom'] =
stock_data['Close'].pct_change(mom_window)
      indicator_data.dropna(inplace=True)
      # If there are records which have no value, e.g momentum for
first record, then they are dropped
      #3 transistion states,for bullish,bearish and sideways markets
respectively, covariance= full to measure correlation ...
      #between parameters and measure variance of parameters, tol is
the percentage error at which we can stop learning
      #and random state ensures that our results are not due to pure
luck
      scaler = StandardScaler()
      scaled_features =
scaler.fit_transform(indicator_data[indicators])
      model = hmm.GaussianHMM(n_components = 3, covariance_type="diag",
n_iter=300, tol=0.001, random_state=50)
      model.fit(scaled_features)
      hidden_states = model.predict(scaled_features)
      #Each regime is essentially a 'state' of a market i.e there is a
state for a bullish,bearish and sideways market
      indicator_data['Regime'] = pd.Series(hidden_states,
indicator_data.index).shift(2) # store predicted regime number for
specific day
      indicator_data['Returns'] = stock_data['Close'].pct_change()#
show daily returns for each day
      regime_returns =
indicator_data.groupby('Regime')['Returns'].mean()# calculates average
return for each regime, we can use this to attach labels to each regime
      worst_state, best_state = regime_returns.idxmin(),
regime_returns.idxmax()
      probs = model.predict_proba(scaled_features)#calculates the
probabilties of being in each regime at each time step(1 day)
      mapping = {regime_returns.iloc[-1]: 'Bull',
regime_returns.iloc[0]: 'Bear', regime_returns.iloc[1]: 'Sideways'}
      indicator_data['Regime_Label'] =
indicator_data['Regime'].map(mapping) # adds a column showing the most
likely regime the stock is in at a specific time

      #assigning column label for all the probabilities of each regime
at each time step and appending to the indicator dataframe
      indicator_data['Position'] = 0
      indicator_data['Strat_Return'] = 0
```

```
      indicator_data.loc[indicator_data['Regime'] == best_state,
'Position'] = 1
      indicator_data.loc[indicator_data['Regime'] == worst_state,
'Position'] = -1
      indicator_data['Strat_Return'] =
indicator_data['Position']*indicator_data['Returns']
      indicator_data['Strat_Equity'] = (1 +
indicator_data['Strat_Return']).cumprod()
      plt.plot(indicator_data['Strat_Equity'])
      plt.title(f"RSI_W: {RSI_window} | vol_w : {vol_window} |
vol_of_vol_w: {vol_of_vol_window} | mom: {mom_window}")
      plt.savefig(os.path.join(save_dir, f'{i}_equity.png'), dpi=300)
      plt.close()


      risk_free_rate = 0.034
      daily_rf = risk_free_rate / 252
      excess_returns = indicator_data['Strat_Return'] - daily_rf
      mean_excess_return = excess_returns.mean()
      std_return = indicator_data['Strat_Return'].std()
      sharpe_ratio = mean_excess_return / std_return * np.sqrt(252)

      final_equity = indicator_data['Strat_Equity'].iloc[-1]

      cumulative_returns = indicator_data['Strat_Equity']
      rolling_max = cumulative_returns.cummax()
      drawdown = cumulative_returns / rolling_max - 1
      max_drawdown = drawdown.min()

      results.append( {
     'ID': i,
    'RSI_window': RSI_window,
    'vol_window': vol_window,
    'vov_window': vol_of_vol_window,
    'mom_window': mom_window,
    'final_equity': final_equity,
    'sharpe_ratio': sharpe_ratio,
    'max_drawdown': max_drawdown
     })


  return results


results = []
seen_Data_Backtest = backtest(N,stock_data, indicator_data, save_dir)

top_20_sharpe = sorted(results, key=lambda x: x['sharpe_ratio'],
reverse=True)[:20]
```

```
top_20_final_equity = sorted(results, key=lambda x: x['final_equity'],
reverse=True)[:20]
top_20_max_drawdown = sorted(results, key=lambda x: x['max_drawdown'],
reverse=True)[:20]

df_sharpe = pd.DataFrame(top_20_sharpe)
df_equity = pd.DataFrame(top_20_final_equity)
df_drawdown = pd.DataFrame(top_20_max_drawdown)

df_sharpe['source'] = 'sharpe'
df_equity['source'] = 'equity'
df_drawdown['source'] = 'drawdown'
all_top20s = pd.concat([df_sharpe, df_equity, df_drawdown],
ignore_index=True)

grouped = all_top20s.groupby(['RSI_window', 'vol_window', 'vov_window',
'mom_window']).size().reset_index(name='count')

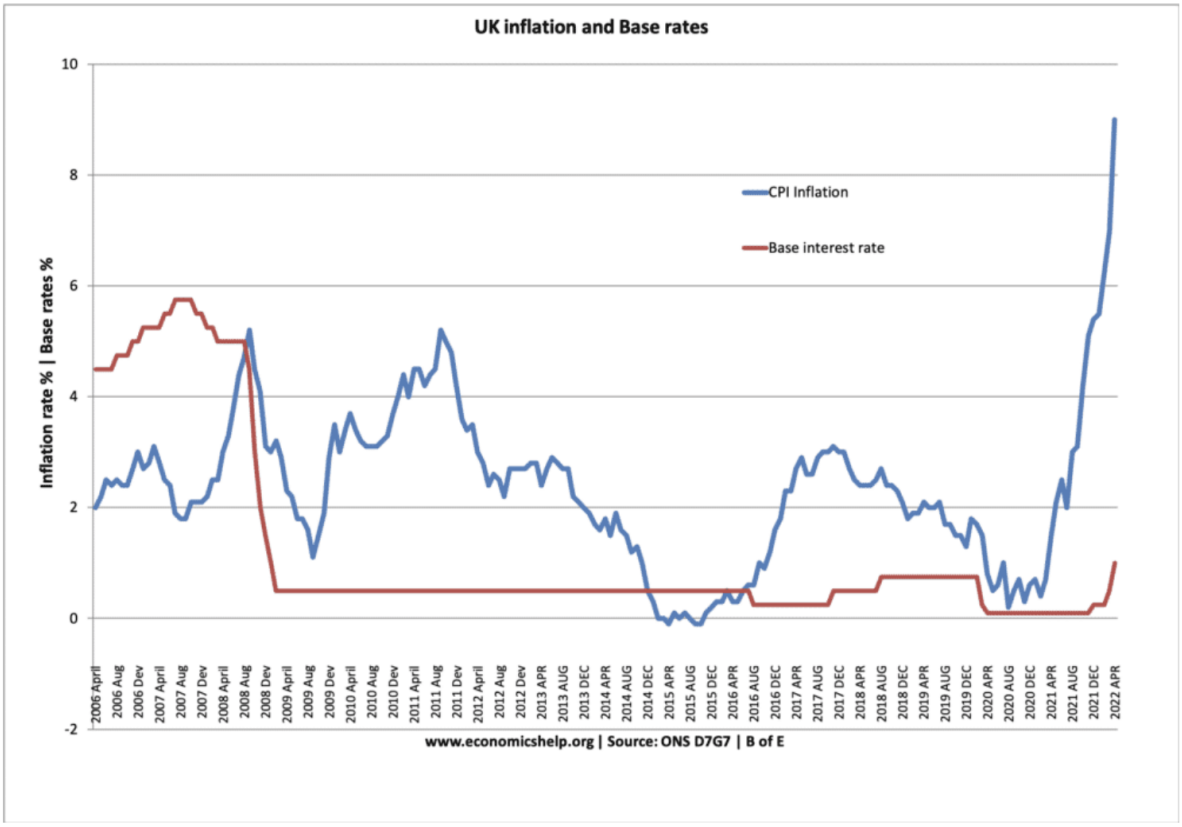
# Sort by most common
most_common = grouped.sort_values(by='count', ascending=False)

print(most_common.head(10))  # Show top 10 most frequent parameter sets
```

Appendix 17-updated output from backtest

```
run index is 40
run index is 50
run index is 60
run index is 70
run index is 80
run index is 90
run index is 100
run index is 110
run index is 120
run index is 130
run index is 140
run index is 150
run index is 160
run index is 170
run index is 180
run index is 190
run index is 200
run index is 210
run index is 220
run index is 230
run index is 240
run index is 250
run index is 260
run index is 270
run index is 280
run index is 290
run index is 300
run index is 310
run index is 320
run index is 330
run index is 340
run index is 350
run index is 360
run index is 370
run index is 380
run index is 390
run index is 400
run index is 410
run index is 420
run index is 430
run index is 440
run index is 450
run index is 460
run index is 470
run index is 480
run index is 490
run index is 500
    RSI_window  vol_window  vov_window  mom_window  count
35         130         140         185         120      3
21          70          90         205         130      3
17          55          85         180          45      3
15          45         230         185          90      3
8           30          35         190         255      3
25          85          90         145         220      2
29         110          85          35         195      2
2            5          75         155         210      2
1            5          55          75          55      2
0            5          40          30         215      2
```

Appenix 18:


UK inflation and Base rates
Inflation rate % | Base rates %
10
8
6
4
2
0
-2
CPI Inflation
Base interest rate
2006 April
2006 Aug
2006 Dec
2007 April
2007 Aug
2007 Dec
2008 April
2008 Aug
2008 Dec
2009 April
2009 Aug
2009 Dec
2010 April
2010 Aug
2010 Dec
2011 April
2011 Aug
2011 Dec
2012 April
2012 Aug
2012 Dec
2013 APR
2013 AUG
2013 DEC
2014 APR
2014 AUG
2014 DEC
2015 APR
2015 AUG
2015 DEC
2016 APR
2016 AUG
2016 DEC
2017 APR
2017 AUG
2017 DEC
2018 APR
2018 AUG
2018 DEC
2019 APR
2019 AUG
2019 DEC
2020 APR
2020 AUG
2020 DEC
2021 APR
2021 AUG
2021 DEC
2022 APR
www.economicshelp.org | Source: ONS D7G7 | B of E